

Mecanum Drive And Controls

This guide explains how the DanceBot mecanum drivetrain works, how the driver controls are mapped, and how those controls become motor power commands in the FTC code.

The code this guide describes is:

TeamCode/src/main/java/org/firstinspires/ftc/teamcode/DanceBotTeleOp.java

What Mecanum Drive Adds

A normal two-wheel or tank-drive robot can drive forward, drive backward, and turn. A mecanum robot can do those things, but it can also move sideways without turning first. This sideways motion is called strafing.

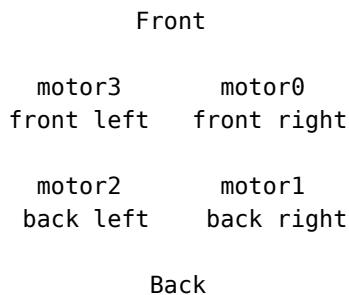
Each mecanum wheel has angled rollers. When all four wheels are powered in the right pattern, the roller forces combine into robot movement:

Robot motion	What the wheels do
Forward/backward	Left and right sides push in the same direction.
Turn	Left and right sides push in opposite directions.
Strafe	Diagonal wheel pairs push together.
Diagonal move	Forward/backward and strafe are mixed together.
Curved move	Forward/backward, strafe, and turn are mixed together.

The important idea is that mecanum drive is not one motor per direction. Every driver command is split across all four wheels.

Wheel Layout

This is the wheel layout used by DanceBotTeleOp when looking down at the robot from above:



The code maps the FTC hardware names to robot positions like this:

Robot position	Hardware name	Hub/port in dancebot.xml
Front right	motor0	Control Hub motor port 0
Back right	motor1	Control Hub motor port 1
Back left	motor2	Control Hub motor port 2
Front left	motor3	Control Hub motor port 3

The split-hub config adds one mechanism motor:

Mechanism	Hardware name	Hub/port in dancebot-split-hubs.xml
Extra motor	motor4	Expansion Hub 2 motor port 0

Driver Controls

DanceBotTeleOp uses gamepad1 for driver control.

Driver input	Code variable	Robot behavior
Left stick up/down	drive	Drive forward/backward
Left stick left/right	turn	Rotate left/right
Right stick left/right	strafe	Slide left/right
Left bumper	scale	Slow mode at 30 percent power
Right bumper	motor4Power	Run optional motor4
No input	all drive powers become 0.0	Robot holds still

FTC gamepads report the left stick Y axis as negative when pushed forward. The code flips that sign so pushing the stick forward means positive drive:

```
double drive = applyDeadZone(-gamepad1.left_stick_y);
```

The left stick X axis is used for turning:

```
double turn = applyDeadZone(gamepad1.left_stick_x);
```

The right stick X axis is used for strafing, but only when all four drive motors are configured:

```
double strafe = fourMotorDrive ? applyDeadZone(gamepad1.right_stick_x) : 0.0;
```

This means the left stick is the main "arcade drive" control and the right stick adds sideways mecanum movement.

Dead Zone

Real joysticks rarely rest at exactly zero. A small amount of drift can make a robot creep when the driver is not touching the controller.

DanceBotTeleOp uses a dead zone of 0.08:

```
private static final double STICK_DEAD_ZONE = 0.08;
```

Any stick value with magnitude 0.08 or less is treated as zero. Values outside that range are rescaled so the robot still reaches full power at the end of the stick travel.

Slow Mode

Slow mode is enabled while the driver holds the left bumper:

```
double scale = gamepad1.left_bumper ? SLOW_SCALE : NORMAL_SCALE;
```

The configured values are:

```
NORMAL_SCALE = 1.00
```

```
SLOW_SCALE   = 0.30
```

Slow mode is useful for lining up with field elements, testing a newly wired robot, or letting new drivers learn without full-speed motion.

The Power Mix

Mecanum control works by combining three driver requests:

```
drive    = forward/backward
```

```
strafe   = left/right slide
```

```
turn     = rotation
```

DanceBotTeleOp calculates the four raw wheel powers like this:

```
double frontLeftPower = drive + strafe + turn;
```

```
double frontRightPower = drive - strafe - turn;
```

```
double backLeftPower = drive - strafe + turn;
```

```
double backRightPower = drive + strafe - turn;
```

Each wheel gets a different combination because each wheel sits in a different corner of the robot and its rollers are angled differently.

Reading The Mix

For pure forward motion:

```
drive = 1.0, strafe = 0.0, turn = 0.0
```

```
front left  = 1.0
```

```
front right = 1.0
```

```
back left   = 1.0
```

```
back right  = 1.0
```

All wheels push the robot forward.

For pure clockwise rotation:

```
drive = 0.0, strafe = 0.0, turn = 1.0
```

```
front left  = 1.0
```

```
front right = -1.0
```

```
back left  = 1.0
back right = -1.0
```

The left side and right side push opposite directions, so the robot turns.

For pure strafe right:

```
drive = 0.0, strafe = 1.0, turn = 0.0
```

```
front left  = 1.0
front right = -1.0
back left   = -1.0
back right  = 1.0
```

The diagonal pairs work together to move the robot sideways.

Normalization

Some joystick combinations can produce raw wheel powers larger than 1.0. Example:

```
drive = 1.0, strafe = 1.0, turn = 1.0
front left = 3.0
```

FTC motor power must stay between -1.0 and 1.0. Instead of clipping only the largest value, the code scales all four wheel powers by the same amount:

```
double maxMagnitude = Math.max(1.0,
    Math.max(Math.max(Math.abs(frontLeftPower), Math.abs(frontRightPower)),
        Math.max(Math.abs(backLeftPower), Math.abs(backRightPower))));
```

Then each wheel is divided by maxMagnitude, multiplied by slow/normal scale, and clipped to the legal motor-power range:

```
frontLeftPower = Range.clip((frontLeftPower / maxMagnitude) * scale, -1.0, 1.0);
frontRightPower = Range.clip((frontRightPower / maxMagnitude) * scale, -1.0,
1.0);
backLeftPower = Range.clip((backLeftPower / maxMagnitude) * scale, -1.0, 1.0);
backRightPower = Range.clip((backRightPower / maxMagnitude) * scale, -1.0, 1.0);
```

Scaling all wheels together preserves the intended direction. The robot may move more slowly, but it still follows the driver's requested mix.

Motor Direction Mapping

The code reverses the right-side motors:

```
frontRight.setDirection(DcMotorSimple.Direction.REVERSE);
frontLeft.setDirection(DcMotorSimple.Direction.FORWARD);
backLeft.setDirection(DcMotorSimple.Direction.FORWARD);
backRight.setDirection(DcMotorSimple.Direction.REVERSE);
```

This is a common drivetrain setup. Motors on opposite sides of the robot are usually mounted as mirror images. If all motors were set to the same direction, one side could drive backward while the other side drives forward.

The direction setting should make this driver expectation true:

Push drive forward -> all four wheels move the robot forward.

If one wheel spins the wrong way during that test, the problem is usually one of these:

Symptom	Likely cause
A physical corner moves when a different motor name is commanded	Motor wires are plugged into different ports than the config expects.
Correct motor moves but forward is backward	That motor direction needs to be reversed in code or the motor is mounted opposite the expected orientation.
Strafing twists hard instead of sliding	One mecanum wheel may be in the wrong physical corner or one diagonal is reversed.
Robot drives diagonally when commanded forward	One wheel is not contributing correctly because of wiring, direction, or mechanical drag.

How We Mapped Code To The Robot

The mapping was built from the Control Hub ports and then corrected to match the physical wheel positions observed on the robot:

```
motor0 -> front right
motor1 -> back right
motor2 -> back left
motor3 -> front left
```

After that, the TeleOp code used one consistent rule:

Calculate desired robot motion first, then send the right power to each named wheel position.

That is why the final call uses wheel positions, not raw motor numbers:

```
setDrivePowers(frontLeftPower, backLeftPower, frontRightPower, backRightPower);
```

Inside setDrivePowers, each value is sent to the motor object that was mapped during initialization:

```
frontLeft.setPower(frontLeftPower);
frontRight.setPower(frontRightPower);
backLeft.setPower(backLeftPower);
backRight.setPower(backRightPower);
```

Keeping this separation matters. Hardware names can be confusing when they are just motor0, motor1, motor2, and motor3. The code should quickly convert those names into robot concepts like frontLeft and backRight, then the rest of the drive math can be written in terms of wheel positions.

Two-Motor Fallback

DanceBotTeleOp can also run if only motor0 and motor1 exist in the hardware configuration:

```
motor0 -> right side  
motor1 -> left side
```

In that mode, fourMotorDrive is false and strafe is forced to zero. The code still drives forward/backward and turns, but it cannot do true mecanum strafing because strafing requires four independently powered wheels.

Optional Motor4

If motor4 exists in the hardware map, the right bumper runs it:

```
double motor4Power = motor4 != null && gamepad1.right_bumper ? scale : 0.0;
```

This is intentionally simple:

```
Right bumper held      -> motor4 runs  
Right bumper released -> motor4 stops
```

That is a safe default for an early mechanism test because the mechanism does not keep moving after the driver releases the button.

Telemetry

The TeleOp loop sends useful values back to the Driver Station:

```
Mode  
Speed  
Turn  
Strafe  
Front Left  
Front Right  
Back Left  
Back Right  
Motor 4  
Battery
```

This lets the team compare what the driver commanded with what the code sent to each wheel. When troubleshooting, put the robot on blocks, move one stick at a time, and read the telemetry while watching the wheels.

Testing Checklist

Use this order when bringing up a mecanum robot.

1. Lift the robot so the wheels cannot drive away.
2. Run DanceBot Motor Test.
3. Confirm each hardware name moves the expected physical wheel:
 - motor0 is front right.
 - motor1 is back right.
 - motor2 is back left.

- motor3 is front left.
4. Run DanceBot TeleOp.
 5. Push the left stick gently forward.
 6. Confirm all wheels try to move the robot forward.
 7. Move the left stick gently left/right.
 8. Confirm the robot would rotate.
 9. Move the right stick gently left/right.
 10. Confirm the robot would strafe.
 11. Hold left bumper and repeat the tests at slow speed.

Only change one thing at a time. If a wheel is wrong, decide whether the problem is the physical port mapping, the wheel position mapping in code, or the motor direction setting.

Common Fixes

If the wrong physical wheel moves for a motor name, fix the hardware map or move the motor wire to the expected port.

If the correct physical wheel moves but spins the wrong direction, fix that wheel's `setDirection` call.

If forward/backward works but strafe is wrong, check the mecanum wheel placement. Viewed from above, the roller pattern should form an X across the robot:

```
front left roller direction  + front right roller direction
back left roller direction   + back right roller direction
```

The four roller angles should visually point toward the center as an X.

If turning works but forward does not, check whether one side is reversed twice: once in wiring or motor mounting, and once in software.

Driver Practice Exercises

After the robot passes the checklist, practice these in slow mode first:

1. Drive forward and stop on a line.
2. Strafe right and stop on a line.
3. Rotate 90 degrees without drifting.
4. Drive diagonally by combining forward and strafe.
5. Drive around a square while keeping the robot pointed in the same direction.
6. Repeat without slow mode.

These exercises teach the difference between robot-centric movement and field positioning before adding sensors or autonomous movement.

What To Improve Next

This TeleOp is robot-centric: pushing the stick forward means "drive toward the robot's front." If the robot turns around, the same stick direction now moves it toward the robot's new front.

The next control upgrade is field-centric drive. Field-centric drive uses an IMU heading to rotate the joystick command so that pushing the stick away from the driver always moves away from the driver, no matter which way the robot is facing.

Before adding field-centric drive, make sure the team understands:

1. The current robot-centric equations.
2. The motor name to wheel position map.
3. The difference between joystick axes and robot axes.
4. How motor direction settings affect the final movement.
5. How to test one wheel at a time before debugging the full drive mix.